Operating system CS2342 – and Technical Writing First Semester 2024		Umm Al-Qura University College of Computing Computer Science and AI Department

Operating system Project

"File Encryption System"

Prepared by:

Name	ID
Rimas Alqurashi	444014714

Report Date:

October 12,2024

Introduction:

data protection is one of the most important challenges in the digital age. Encryption is a secure way to ensure the confidentiality of private information and can secure files from hackers and data thieves who can gain unauthorized access to files and data within networks and computing. File encryption systems are a technology for controlling personal data by converting it into a format that can only be read by authorized people or organisation systems.

Some aim of file encryption is to:

- Data protection by Prevent unauthorized access to files.
- Privacy protection by Ensure the privacy of various information.

The purpose of this report is to understand the working of the encryption algorithm and how to apply it in C language to files and to show the file encryption system that encrypts and decrypts the file content using the XOR algorithm.

Project Requirements:

- Programming languages : " **C** ".
- Files: There must be a text file containing the text to be encrypted.
- Libraries: **<stdio.h> , <stdlib.h>**

What is XOR algorithm? And how it works?

XOR is a logical operation short for "EXCLUSIVE OR" that takes two values "0,1" and produces a new value.

XOR states that if the two values are equal, the result is 0, but if they are different, the result is 1.

Encryption process using XOR:

- If we want to encrypt a letter, we must first convert it to its digital value in the binary system.

- choose the value of the key "used in encryption" Some points must be taken into account when choosing the key like:
    a. random and unexpected
    b. it must be long enough to be secure, a short key such as one letter exposes you to attacks
    c. not repeating the key and so on.

- Then we do the XOR operation. If the first digit of the literal number that we converted to binary is equal to the first digit of the key number, we put "0" and if they are different, we put "1" and we apply this operation to all the numbers and the result of the XOR will appear and we convert it to its real value, which represents a different letter in ASCII.

Example of Encryption& Decryption process:

"where A represents the letter and K represents the key"

```
(A)  01000001
                ^
(K)  01001011
------------
(Results of XOR) = 00001010
```

Which represents the number 10 and represents a different letter in ASCII.

To decrypt, we take the result that appeared from the encryption process: "00001010" and perform the XOR process again with the key. The result will appear: 65, which represents the letter 'A.'

```
(result from encryption )  00001010
                                   ^
        (K)                01001011
                           ------------
                       (A)  01000001
```

Code:

First, we open a new page in the Text editor in Ubuntu.
Then we write the code in a text editor and save it with the C extension.

```c
#include <stdio.h>  // مكتبة الإدخال والإخراج
#include <stdlib.h> //: مكتبة الدوال العامة زي
                    // exit و malloc

                // ميثود للتشفير وفك التشفير لمحتوى الملف //
void encryptDecrypt(char *inputFile, char *outputFile, char key) {
    (بصيغة باينري) نفتح ملف الإدخال للقراءة //
    FILE *inFile = fopen(inputFile, "rb");
   (بصيغة باينري) نفتح ملف الاخراج للقراءة //
    FILE *outFile = fopen(outputFile, "wb");
                نتأكد إن الملفات فتحوا صح //
    if (inFile == NULL || outFile == NULL) {
        printf("Error opening files.\n"); //لو فيه مشكلة في الفتح، نطبع رسالة خطأ
        exit(1); // نخرج من البرنامج
    }

    char ch;// متغير عشان نخزن كل حرف بنقرأه
نقرأ كل حرف من ملف الإدخال لحد ما نوصل لنهاية الملف //
    while ((ch = fgetc(inFile)) != EOF) {
        fputc(ch ^ key, outFile)  // نطبق عملية XOR مع المفتاح ونكتب النتيجة في ملف الإخراج
    }
// نقفل ملف الإدخال والاخراج //
    fclose(inFile);
    fclose(outFile);put
}

int main() {
    char *inputFile = "input.txt";  // اسم ملف الإدخال
    char *outputFile = "output.txt"; // اسم ملف الإخراج
    char key = 'K';                  // مفتاح التشفير

        // نستدعي الميثود عشان نشفر الملف
    encryptDecrypt(inputFile, outputFile, key);
    printf("File encrypted successfully.\n"); // نطبع رسالة ان التشفير نجح

    // لفك التشفير، يمكن استدعاء الميثود مرة أخرى بنفس المفتاح
    encryptDecrypt(outputFile, "decrypted.txt", key);
    printf("File decrypted successfully.\n");  // نطبع رسالة ان فك التشفير نجح

    return 0; // نخرج من البرنامج اذا نجح
}
```

- We used the library "**<stdio.h>** " which we use in the material labs and which is used for input and output operations such as printing, reading, etc.

- The other library "**<stdlib.h>** " contains a set of useful general functions. We used it for the `**exit**` function: It is used to terminate the program and in case of an error such as not being able to open files, it can close the program immediately and correctly. It can take a value that expresses the state of the program termination (such as 0 for success and 1 for failure).

- `**encryptDecrypt**` function: Responsible for the encryption and decryption process of file content using the XOR algorithm:
    a) Takes the name of the file "input.txt"containing the text to be encrypted to read from it.

    b) It also takes the name of the file "output.txt"c that contains the text after it has been encrypted to write the encrypted text in it .

    c) It also takes the encryption key that will be used in the xor process.

    d) `fopen` is used to open input and output files.

    e) Checks the opening if any of the files fail to open, displays an error message and uses `exit` to terminate the program.

    f) Encryption/Decryption Process: we use `while` loop to read each character from the input file using `fgetc` until reaching the end of the file and thenWe apply the XOR operation between each character and the key, then write the result to the output file using `fputc`.

    g) After the encryption and decryption operations are completed, the files are closed using `fclose`.

- **‘Main’** function : It is the main provider from which the program starts.
    a) From the main we specify the names of the input file, like (`input.txt`) and the output file, like (`output.txt`) and we also specify the value of the encryption key. In our code we used the key "K".

    b) Then we call the `encryptDecrypt` function to encrypt the content of the input file and write the result in the **‘output.txt’** file.

    c) If the process is successful, a message will appear stating that the encryption was successful.

    d) To decrypt we call the `encryptDecrypt` function again and the original text after it decrypted will be written to the file `decrypted.txt.`

    e) If the process is successful, a message will appear stating that the decryption was successful.

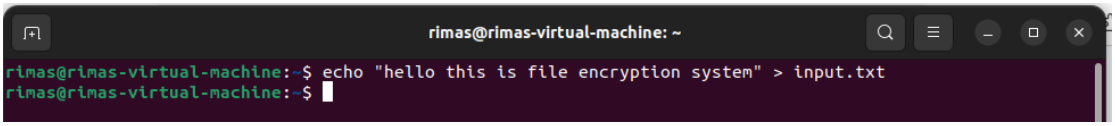
    f) Finally `main` will return 0 using the 'exsit' function which means the program completed successfully.

Notes:

- ✓ You must create a text file like `input.txt` containing the text to be encrypted before running the program. We used the echo command in the terminal and we will discuss it.
- ✓ You can change the encryption key or file names as needed.
- ✓ The file "output.txt" is created when calling the function `**encryptDecrypt**` for encryption and also the file "decrypted.txt" is created when calling the function `**encryptDecrypt**` for decryption.

Terminal output:

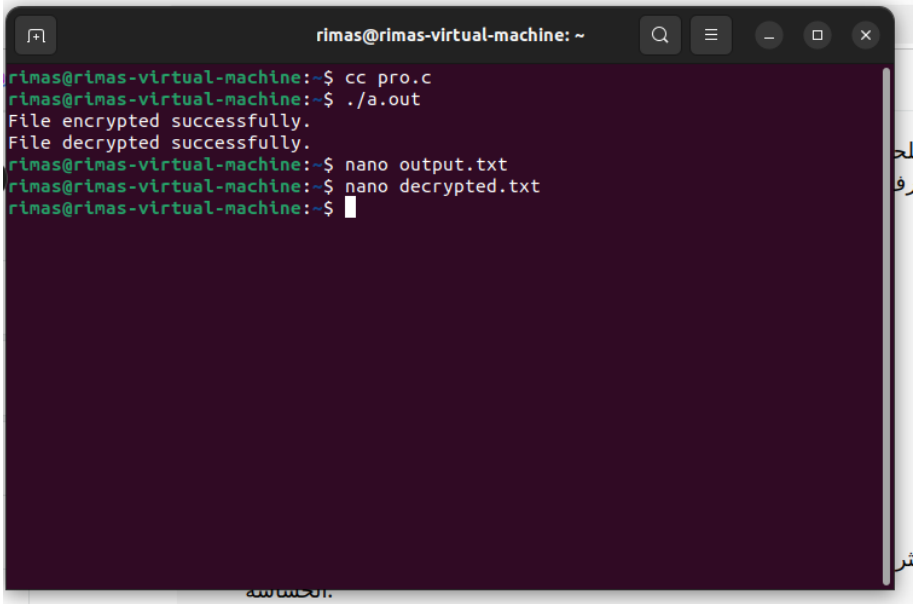
1. We create a text file to write the text to be encrypted in it "we used echo command to create file text".



2. In the terminal image below, first we compile using the "cc" command, then write the file name with the C extension like that:

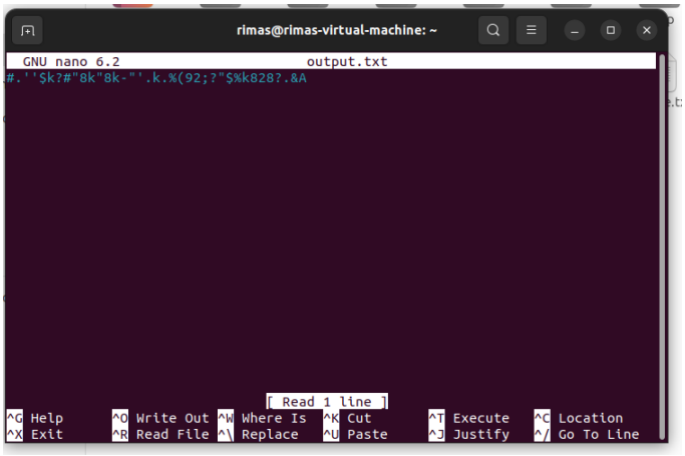
"cc pro.c"

3. To display the result we used the command **"./a.out"**, We notice that a phrase was printed indicating the success of the encryption/decryption process.



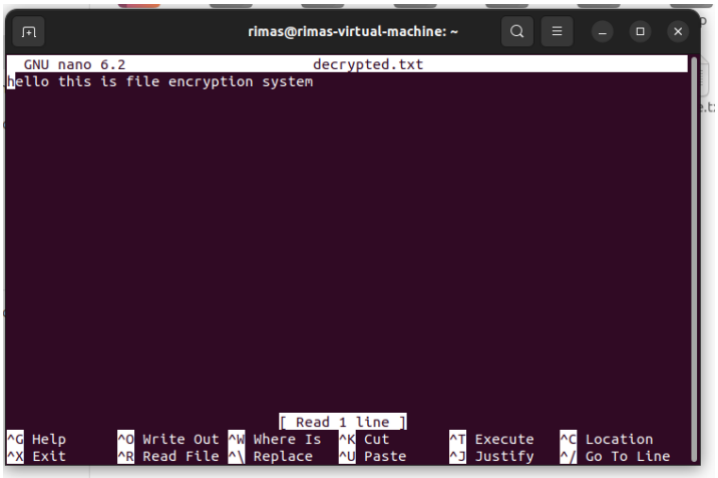
4. After executing the code, Now we have a file was created containing the encrypted text with name “output.txt”. After we run the code and we want to display the encrypted text inside it, we use the command **“nano output.txt”**.

encrypted text:



5. Also, another file was created when we run the code with name "decrypted.txt" which displays the original text after it decrypted. We use the command **"nano decrypted.txt"** to display the original text inside the decrypted file.

Decrypted text:



In conclusion..

This project reflects the basic concepts of encryption using the XOR algorithm and is a first step in understanding more complex encryption techniques. We hope to improve this system to make it more secure and efficient in practical applications.

References:

- https://www.techtarget.com/searchsecurity/definition/Encrypting-File-System

- https://blog.box.com/what-is-file-encryption

- https://academy.hsoub.com/programming/c/-أدوات-مكتبة-stdlib--في-لغة-سي-c-r1797/

- Poe: For help and clarification and suggestions in code and reports .